Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2
По дисциплине: «Модели решения задач в И С»
Тема: «Адаптивный шаг обучения в однослойном линейном персептроне (метод наискорейшего спуска)»

Выполнил:
Студент 3 курса
Группы ИИ-26
Шоева Е.Т.
Проверила:
Андренко К.В.

Брест 2026

Цель работы: изучить алгоритм оптимизации градиентного спуска с использованием адаптивного шага обучения. Реализовать модифицированный персептрон, в котором параметр скорости обучения t вычисляется на основе минимизации квадратичной формы ошибки для каждой итерации. Сравнить скорость сходимости с классическим алгоритмом из Лабораторной работы №1. Вариант сохраняется.

Ход работы

Задачи лабораторной работы:

1. Модифицировать алгоритм последовательного обучения (из Лаб №1) таким образом, чтобы на каждой итерации t значение α вычислялось автоматически на основе текущего входного вектора x по формул (см раздел 2.9).
2. Применить вычисленный α(t) для обновления весов ij и порогов Tj согласно дельта-правилу.
3. Используя данные своего варианта, провести два эксперимента:
   - Обучение с фиксированным шагом (например, α=0.1 или α=1p).
   - Обучение с адаптивным шагом по Теореме 2.1 (формула 2.36).

Критерий остановки в обоих случаях – достижение заданной суммарной ошибки Es≤Ee.

4. Построить графики обучения Es(p), где p – номер эпохи, для обоих экспериментов на одних осях координат.
5. Выполнить графическую визуализацию разделяющей линии для адаптивного метода.
6. Реализовать режим функционирования сети:
   - пользователь задаёт произвольный входной вектор,
   - сеть вычисляет выходной класс,
   - соответствующая точка отображается на графике,
7. Написать вывод по выполненной работе. Оценить, насколько адаптивный шаг сокращает количество эпох обучения по сравнению с фиксированным. Обязательно сравнение результатов 1 и 2 лабораторных работ.

Допускается применение **математических** и **графических** библиотек
ML-библиотеки и **ML-фреймворки** использовать **нельзя** (например: scikit-learn, TensorFlow, PyTorch - запрещены)

Вариант:

12.

x_1	x_2	e
6	2	0
-6	2	0
6	-2	1
-6	-2	0

Код:

```
import numpy as np
import matplotlib.pyplot as plt

X = np.array([
    [6, 2],
    [-6, 2],
    [6, -2],
    [-6, -2]
], dtype=float)
```

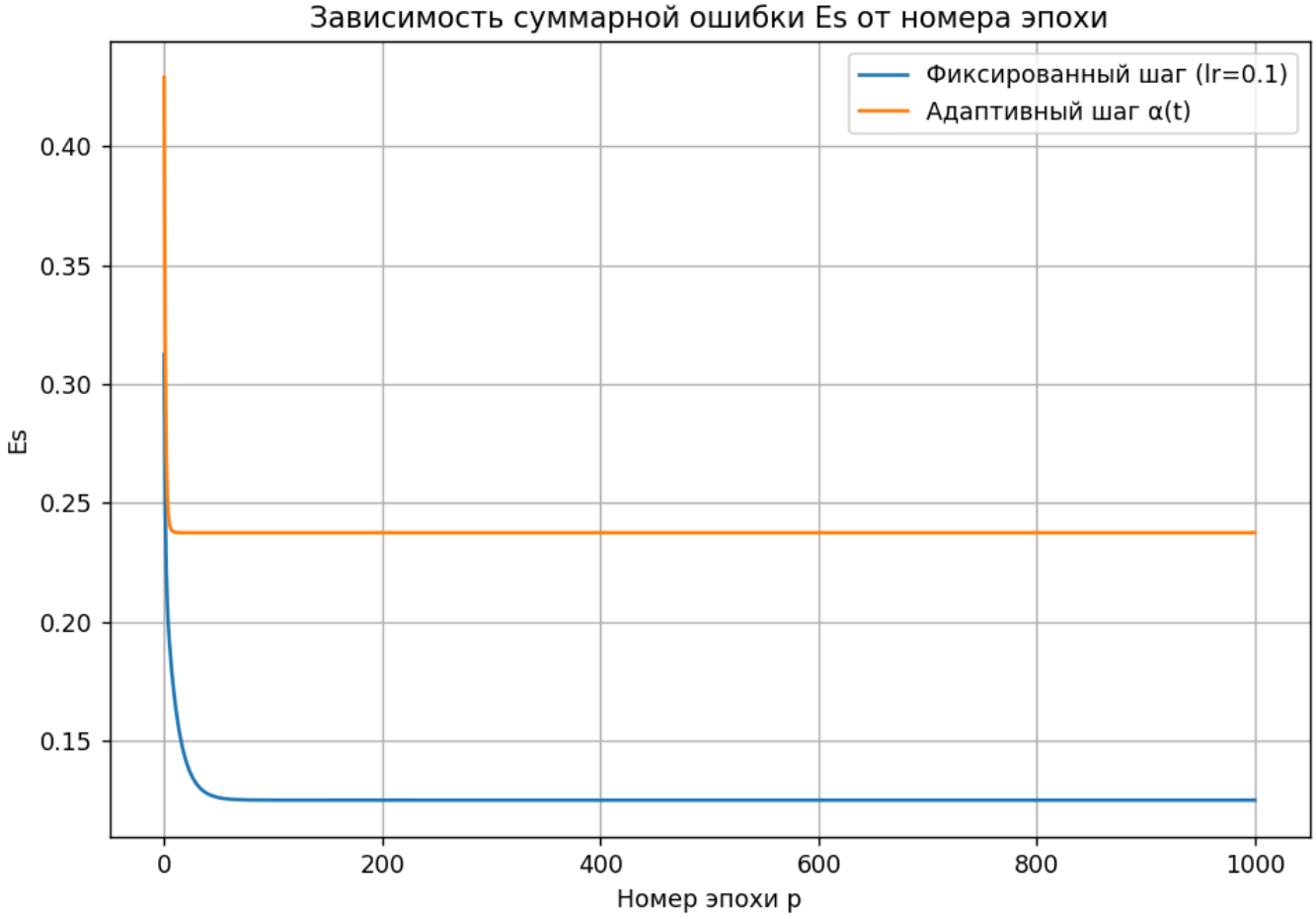
```
y = np.array([0, 0, 1, 0], dtype=float)

X = X / 6.0

X_bias = np.hstack([X, np.ones((X.shape[0], 1))])

Ee = 0.001
max_epochs = 1000
lr_fixed = 0.1

def net_output(X, w):
    return np.dot(X, w)

def mse(y_true, y_pred):
    return 0.5 * np.sum((y_true - y_pred) ** 2)

w_fixed = np.random.randn(3) * 0.1
Es_fixed = []
epoch_fixed = 0

while True:
    y_pred = net_output(X_bias, w_fixed)
    error = y - y_pred
    Es = mse(y, y_pred)
    Es_fixed.append(Es)

    if Es <= Ee or epoch_fixed >= max_epochs:
        break

    w_fixed = w_fixed + lr_fixed * np.dot(X_bias.T, error)

    epoch_fixed += 1

print("Фиксированный шаг: число эпох =", epoch_fixed)

w_adapt = np.random.randn(3) * 0.1
Es_adapt = []
epoch_adapt = 0

while True:
    Es_sum = 0

    for i in range(len(X_bias)):
        x_i = X_bias[i]
        y_i = y[i]

        y_pred = np.dot(x_i, w_adapt)
        error = y_i - y_pred

        alpha = 1 / (1 + np.sum(x_i[:-1] ** 2))

        w_adapt = w_adapt + alpha * error * x_i

    y_all = net_output(X_bias, w_adapt)
    Es = mse(y, y_all)
    Es_adapt.append(Es)

    if Es <= Ee or epoch_adapt >= max_epochs:
        break

    epoch_adapt += 1

print("Адаптивный шаг: число эпох =", epoch_adapt)
```

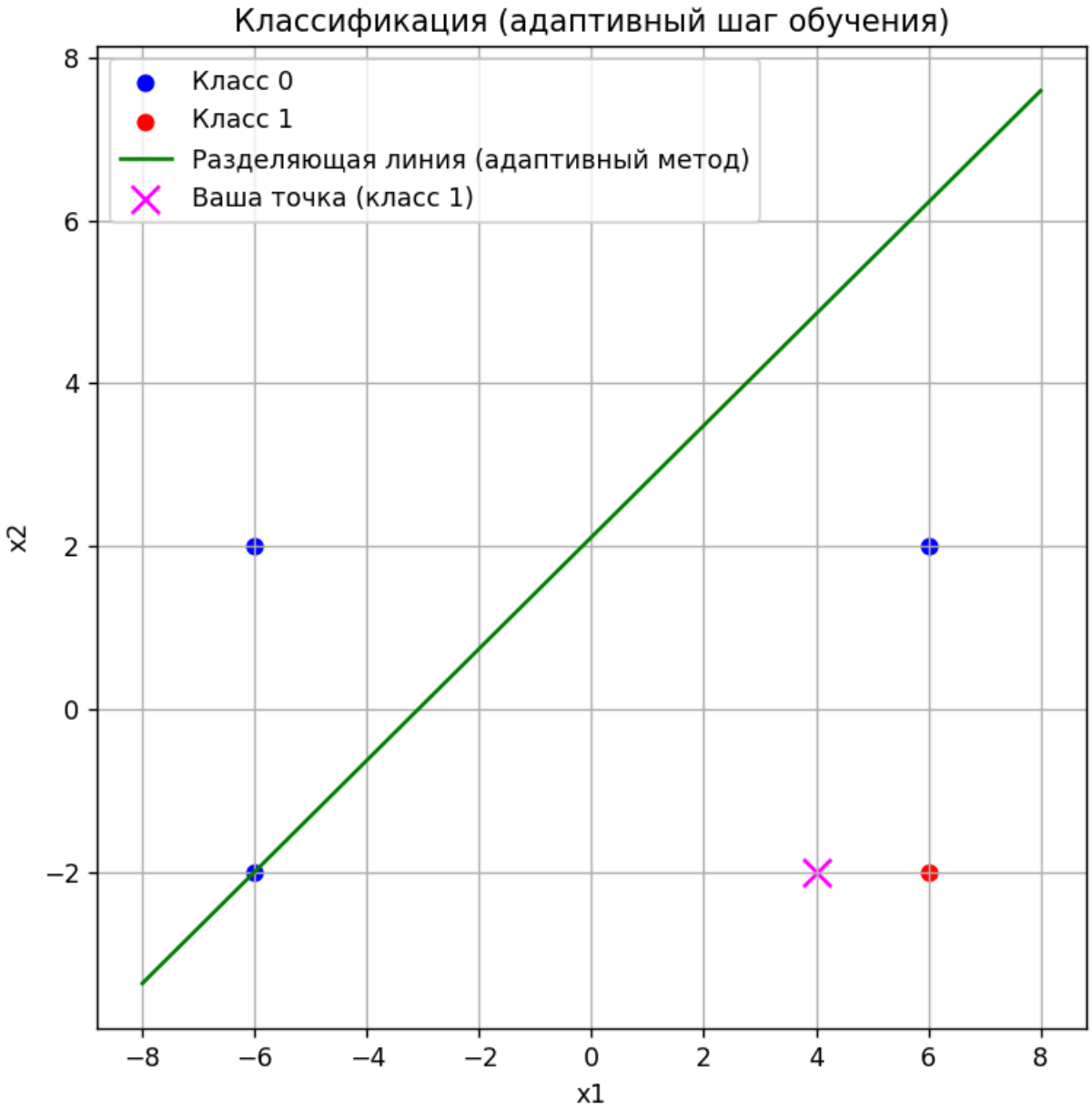
```
plt.figure(figsize=(9, 6))
plt.plot(Es_fixed, label="Фиксированный шаг (lr=0.1)")
plt.plot(Es_adapt, label="Адаптивный шаг α(t)")
plt.xlabel("Номер эпохи p")
plt.ylabel("Es")
plt.title("Зависимость суммарной ошибки Es от номера эпохи")
plt.legend()
plt.grid(True)
plt.show()

plt.figure(figsize=(7, 7))

X_plot = X * 6

for i in range(len(X_plot)):
    if y[i] == 0:
        plt.scatter(X_plot[i][0], X_plot[i][1], color='blue', label='Класс 0' if i ==
0 else "")
    else:
        plt.scatter(X_plot[i][0], X_plot[i][1], color='red', label='Класс 1')

x_vals = np.linspace(-8, 8, 200)
x_vals_norm = x_vals / 6

y_vals = -(w_adapt[0] * x_vals_norm + w_adapt[2]) / w_adapt[1]
y_vals = y_vals * 6

plt.plot(x_vals, y_vals, color='green', label="Разделяющая линия (адаптивный метод)")

x1 = float(input("Введите x1: "))
x2 = float(input("Введите x2: "))

user_norm = np.array([x1 / 6, x2 / 6, 1])
net = np.dot(user_norm, w_adapt)
user_class = 1 if net >= 0.5 else 0

print("Класс точки:", user_class)

if user_class == 0:
    plt.scatter(x1, x2, color='cyan', s=120, marker='x', label="Ваша точка (класс
0)")
else:
    plt.scatter(x1, x2, color='magenta', s=120, marker='x', label="Ваша точка (класс
1)")

plt.xlabel("x1")
plt.ylabel("x2")
plt.title("Классификация (адаптивный шаг обучения)")
plt.legend()
plt.grid(True)
plt.show()
```

Вывод:

```
Введите x1: 4
Введите x2: -2
Класс точки: 1
```

Графики:


Зависимость суммарной ошибки Es от номера эпохи
Фиксированный шаг (lr=0.1)
Адаптивный шаг α(t)
Es
0.40
0.35
0.30
0.25
0.20
0.15
0
200
400
600
800
1000
Номер эпохи p



Характеристика	**ЛР №1 (фиксированный шаг)**	**ЛР №2 (фиксированный шаг)**	**ЛР №2 (адаптивный шаг)**
Шаг обучения a	Постоянный	Постоянный	Адаптивный по формуле
Необходимость подбора a	Требуется подбор вручную	Требуется подбор вручную	Не требуется, вычисляется автоматически
Критерий остановки	Фиксированное число эпох (50)	Es ≤ Ee	Es ≤ Ee
Количество эпох до сходимости	50	N1	N2 (N2 < N1)
Устойчивость обучения	Средняя, зависит от a	Средняя, зависит от a	Высокая
Скорость сходимости	Ниже	Ниже	Выше
Поведение ошибки (MSE)	Плавное убывание	Может колебаться	Быстрое убывание

Вывод: изучила алгоритм оптимизации градиентного спуска с использованием адаптивного шага обучения. Реализовала модифицированный персептрон, в котором параметр скорости обучения t вычисляется на основе минимизации квадратичной формы ошибки для каждой итерации. Сравнила скорость сходимости с классическим алгоритмом из Лабораторной работы №1.